Trabajo Práctico Grupal – Semana 3

Fase 3: Game Loop Completo, Prefabs Dinámicos, UI, Corrutinas y Game Juice

> **FECHA LÍMITE DE ENTREGA OBLIGATORIA**
>
> **LUNES 9 DE NOVIEMBRE DE 2026 HASTA LAS 23:50 HS.** Entrega exclusiva mediante **Pull Request en GitHub** hacia el repositorio oficial del docente. No se recibirán proyectos por correo ni ningún otro canal.

1. ¿De qué se trata la Semana 3? (En palabras simples)

En la Semana 1 creamos las clases lógicas puras en C# (vidas, daño, stats). En la Semana 2 las conectamos a Unity para que el personaje se mueva con físicas reales (`Rigidbody2D` con `linearVelocity`) y choque o active sensores (`Colliders` y `Triggers`). Ahora, en la **Semana 3**, convertimos ese prototipo rígido en un **videojuego jugable de principio a fin**.

¿Qué significa esto? Que el juego tiene que poder generar cosas en el aire (balas, enemigos o monedas con `Instantiate`), manejar el paso del tiempo con esperas sin congelar la pantalla (`Corrutinas`), mostrarle al jugador lo que pasa en la pantalla mediante una interfaz gráfica (`Canvas`: barra de vida y textos), sonar cuando hay golpes o monedas (`AudioSource`) y permitir reiniciar o cambiar de nivel limpiamente (`SceneManager`).

2. Nuevos Ejes Temáticos Obligatorios de Unity a Integrar

- **Prefabs e Instanciación Dinámica:** Generar proyectiles, ítems o enemigos en tiempo de ejecución con `Instantiate()` y destruirlos con `Destroy(gameObject, tiempo)` para no saturar la memoria RAM.
- **Estructuras de Datos (Arrays y `List<T>`):** Usar arreglos fijos (ej. waypoints de patrulla de enemigos) y listas dinámicas (ej. enemigos activos en pantalla o inventario dinámico recolectado).
- **Manejo de Tiempo con Corrutinas (`IEnumerator` y `yield return`):** Pausas no bloqueantes para cuadros de invulnerabilidad tras recibir daño (parpadeo visual del sprite), cadencia de disparo o recarga.
- **Interfaz de Usuario 2D (UI Canvas):** Visualizar la salud del personaje con un `Slider` y textos con `TextMeshProUGUI` (vidas, puntaje o munición). Prohibido usar el viejo componente `Text` deprecado.
- **Audio, Control de Escenas y "Game Juice":** Reproducir sonidos con `audioSource.PlayOneShot()`, implementar sacudida de pantalla (*Camera Shake*) ante impactos fuertes y reiniciar nivel con `SceneManager.LoadScene()`.

3. Metodología Rotativa: Tercer Sorteo de Repositorios (Code Handover)

Nuevamente cambiamos de proyecto. Heredarán el repositorio que trabajó otro equipo en la Semana 2. Su trabajo es:

1. **Auditar lo recibido:** Revisar si compila, si tiene físicas estables y si respetaron la clase pura de C#.
2. **Refactorizar y corregir deudas técnicas:** Si algo está roto, incompleto o no compila, lo arreglan.
3. **Construir la Fase 3 encima:** Agregar los nuevos módulos sin borrar la lógica anterior.

Grupo	Tema Semana 1 (POO)	Tema Semana 2 (Físicas)	NUEVO TEMA SEMANA 3 (Fase Final)
Grupo 1	Tema 1 (Mario)	Tema 3 (Pokémon)	**Tema 5: DOOM 2D**
Grupo 2	Tema 2 (Zelda)	Tema 1 (Mario)	**Tema 7: Dark Souls 2D**
Grupo 3	Tema 3 (Pokémon)	Tema 2 (Zelda)	**Tema 4: Street Fighter 2D**
Grupo 4	Tema 4 (Street Fighter)	Tema 5 (DOOM)	**Tema 9: Hollow Knight 2D**
Grupo 5	Tema 5 (DOOM)	Tema 9 (Hollow Knight)	**Tema 2: The Legend of Zelda 2D**
Grupo 6	Tema 6 (Minecraft)	Tema 4 (Street Fighter)	**Tema 10: Pac-Man 2D**
Grupo 7	Tema 7 (Dark Souls)	Tema 10 (Pac-Man)	**Tema 6: Minecraft 2D**

Grupo	Tema Semana 1 (POO)	Tema Semana 2 (Físicas)	NUEVO TEMA SEMANA 3 (Fase Final)
Grupo 8	Tema 8 (Resident Evil)	Tema 6 (Minecraft)	**Tema 1: Super Mario Bros 2D**
Grupo 9	Tema 9 (Hollow Knight)	Tema 8 (Resident Evil)	**Tema 3: Pokémon 2D**
Grupo 10	Tema 10 (Pac-Man)	Tema 7 (Dark Souls)	**Tema 8: Resident Evil 2D**
Grupo 11	Tema 1 (Mario B)	Tema 8 (Resident Evil B)	**Tema 5: DOOM 2D (Modo Horda/Supervivencia)**

4. La Lista de 7 Pasos Indispensables para Esta Entrega

Para no perderse entre tantos conceptos, cada equipo debe completar esta secuencia de ingeniería:

1. **Sincronizar el repositorio:** Actualizar el Fork con el upstream del docente e ingresar a la carpeta del nuevo tema asignado.
2. **Auditoría inicial:** Probar la escena heredada. Si tiene errores de compilación, solucionarlos de inmediato.
3. **Prefab con Spawn Dinámico:** Crear al menos un Prefab (proyectil, ítem recolectable o enemigo) y dispararlo/instanciarlo mediante código C# usando `Instantiate()` en una posición fija o punto emisor (`firePoint`).
4. **Invulnerabilidad temporal con Corrutina:** Cuando el personaje sufra daño, activar una corrutina con `yield return new WaitForSeconds(0.1f)` que haga parpadear el `SpriteRenderer` y prevenga recibir golpes dobles instantáneos.
5. **Construcción del UI Canvas:** Crear un Canvas con un `Slider` que represente la salud (conectada al modelo POO) y un `TextMeshProUGUI` que muestre el estado, munición o puntaje.
6. **Game Juice & Audio:** Incorporar un `AudioSource` que ejecute un sonido con `PlayOneShot()` ante una acción clave (disparo, daño, moneda) y un ligero temblor de cámara (Camera Shake).
7. **Condición de Fin y Reinicio:** Si la vida del modelo llega a cero, mostrar cartel de derrota y reiniciar el nivel tras 2 segundos utilizando `SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex)`.

5. Reglas Técnicas de C# y Código Moderno (¡Cero Advertencias!)

> **Checklist Antierrores para la Corrección Docente**
>
> - **Prohibido usar `rb.velocity`:** Usar obligatoriamente **`rb.linearVelocity`**. En versiones modernas de Unity, `velocity` queda obsoleta.
> - **Prohibido usar `UnityEngine.UI.Text`:** Utilizar exclusivamente **`TextMeshProUGUI`** importando `using TMPro;`.
> - **No a las "God Classes":** Las clases puras de C# de la Semana 1 (modelos de daño, inventario, fórmulas) NO deben heredar de `MonoBehaviour`. Siguen viviendo como objetos instanciados con `new` dentro de `Awake()`.
> - **Destrucción programada:** Todo proyectil o partícula instanciada debe autodestruirse mediante `Destroy(gameObject, tiempoMaximo)` en su método `Start()` para evitar fugas de memoria.

6. Flujo de Trabajo Git y Formato del Reporte Técnico

- **Ramas individuales obligatorias:** Ningún alumno puede commitear en `main`. Trabajar en ramas individuales: `dev-NombreApellido-Legajo` (ej. `dev-LucasGomez-56201`).
- **Pull Request de entrega:** El Líder unifica las ramas y abre un Pull Request hacia la cátedra titulado estrictamente: `[ENTREGA FASE 3] Grupo XX - Tema: NombreDelJuego`
- **Archivo `reporte.md` obligatorio en la raíz:**
  1. Lista de integrantes (Nombre, Apellido y Legajo).
  2. Auditoría del código heredado de la Semana 2: qué recibieron roto o incompleto y qué refactorizaron.
  3. Explicación de las corrutinas implementadas y el sistema de Prefabs instanciados.
  4. Captura o descripción de la UI configurada (Slider de vida y TextMeshPro).
  5. Guía paso a paso para que el docente pruebe la escena al darle Play.

7. Criterios de Evaluación Ponderados

Criterio Evaluado	Descripción del Logro Técnico	Puntaje
Instanciación y Ciclo de Prefabs	Uso correcto de `Instantiate()` y limpieza con `Destroy()` sin generar basura en memoria RAM.	**25%**
Corrutinas y Manejo de Tiempo	Invulnerabilidad no bloqueante, efectos visuales de parpadeo y temporizadores con `IEnumerator`.	**20%**
Integración de UI (Canvas y TMP)	Sincronización en tiempo real del modelo de datos con el `Slider` y textos de `TextMeshProUGUI`.	**20%**
Game Juice, Audio y SceneManager	Feedback de impacto (sonido con `PlayOneShot`, cámara responsiva) y reinicio funcional de partida.	**15%**
Auditoría, Refactorización y Git	Resolución de deudas del grupo anterior, uso estricto de branches por legajo y calidad del `reporte.md`.	**20%**

8. Los 10 Temas Explicados: ¿Qué debe funcionar exactamente en pantalla?

A continuación se detalla lo que el docente va a evaluar al abrir el proyecto de cada grupo y darle **Play**. No se pide un juego gigante; se pide que los siguientes elementos interactúen con precisión:

Tema 1: Super Mario Bros 2D (Asignado a: Grupo 8)

- **Prefabs e Instanciación:** Al golpear un Bloque de Pregunta desde abajo, se instancia dinámicamente un Prefab de Moneda o Champiñón con un pequeño impulso vertical.
- **Corrutinas:** Si Mario recibe daño de un Goomba, activa una corrutina de 2 segundos de invulnerabilidad donde el sprite parpadea entre transparente y opaco, ignorando nuevos golpes durante ese lapso.
- **UI & Audio:** Texto `TextMeshProUGUI` que muestra `Monedas: X` y `Vidas: Y`. Sonido clásico de recolección al agarrar la moneda. Si vidas llega a 0, reinicio de escena con `SceneManager`.

Tema 2: The Legend of Zelda 2D (Asignado a: Grupo 5)

- **Prefabs e Instanciación:** Link puede disparar un rayo de espada o flecha (Prefab con `linearVelocity`) instanciado en su posición al presionar la tecla de ataque si su salud está al máximo.
- **Estructuras de Datos:** Array de `Transform[]` en la mazmorra donde patrulla un enemigo esqueleto pasando ordenadamente del Punto A al Punto B y luego al C.
- **UI & Audio:** Slider o contenedor de corazones en el Canvas sincronizado con `Heroe.Salud`. Sonido al impactar la flecha contra una pared o enemigo.

Tema 3: Pokémon 2D (Asignado a: Grupo 9)

- **Prefabs e Instanciación:** Al pisar el pasto alto y activarse el combate, se instancia dinámicamente el Prefab del Pokémon salvaje frente al entrenador.
- **Corrutinas:** La animación de transición de batalla utiliza una corrutina para pausar el control del personaje durante 1 segundo antes de habilitar el menú de combate.
- **UI & Audio:** Panel Canvas con dos barras de vida (Sliders): la de tu Pokémon y la del rival. Texto con el nombre de la criatura. Sonido de grito o ataque al golpear.

Tema 4: Street Fighter 2D (Asignado a: Grupo 3)

- **Prefabs e Instanciación:** Al presionar la tecla de ataque especial, se descuenta Ki en la clase lógica y se instancia un Prefab `Hadoken` que avanza en línea recta a velocidad constante.
- **Game Juice & Corrutinas:** Cuando un luchador recibe un golpe fuerte, se produce un *Hitstop* (micro-pausa temporal de 0.08 segundos con corrutina) y una sacudida de pantalla (Camera Shake).
- **UI & Audio:** Dos Sliders gigantes en la parte superior que reflejan la vida de ambos combatientes. Sonido de impacto al conectar el golpe.

Tema 5: DOOM 2D (Asignado a: Grupo 1 y Grupo 11)

- **Prefabs e Instanciación:** Disparo con click izquierdo que instancia proyectiles de Escopeta o Plasma en la posición de la boquilla (`firePoint`) con dispersión angular leve.
- **Listas Dinámicas (`List<GameObject>`):** El gestor de nivel mantiene una lista de demonios activos. Cada vez que un demonio muere, se elimina de la lista con `.Remove()`.
- **UI, Audio & Muerte:** Slider de salud y texto con contador de munición. Sonido de disparo contundente. Si los demonios acaban con el Doomguy, la escena se recarga automáticamente.

Tema 6: Minecraft 2D (Asignado a: Grupo 7)

- **Prefabs e Instanciación:** Al picar un bloque y agotarse su dureza, el bloque se destruye y se instancia en su lugar un Prefab "Drop" miniatura con física liviana que cae al suelo.
- **Corrutinas:** Tiempo de picado continuo gestionado con una corrutina que muestra el desgaste progresivo del bloque antes de romperse.
- **UI & Audio:** Barra de durabilidad de la herramienta (Slider) y panel con texto del inventario acumulado. Sonido de piedra rompiéndose al completarse el minado.

Tema 7: Dark Souls 2D (Asignado a: Grupo 2)

- **Prefabs e Instanciación:** El jefe de la arena dispara un ataque de proyectil de fuego en área (Prefab con `CircleCollider2D` Is Trigger) que se expande por el suelo.
- **Corrutinas & I-Frames:** La mecánica de rodamiento (Roll) utiliza una corrutina que otorga 0.4 segundos de invulnerabilidad cambiando la capa del jugador (evitando daño) y consumiendo estamina en el modelo.
- **UI & Audio:** Dos barras en Canvas: Barra verde para Estamina y barra roja para Salud. Sonido de campana o fuego al descansar en la Hoguera para reiniciar enemigos.

Tema 8: Resident Evil 2D (Asignado a: Grupo 10)

- **Prefabs e Instanciación:** Al abatir a un Zombie, existe una probabilidad de instanciar dinámicamente un Prefab de munición o hierba curativa en sus coordenadas.
- **Corrutinas:** Estado de envenenamiento o mordida: si el zombie agarra al protagonista, una corrutina aplica daño recurrente cada 1 segundo hasta que el jugador se libere o consuma un spray.
- **UI, Audio & Juice:** Texto indicador de estado médico (`Fine` en verde, `Caution` en amarillo, `Danger` en rojo). Sonido de gemido de zombie y recarga de arma.

Tema 9: Hollow Knight 2D (Asignado a: Grupo 4)

- **Prefabs & Game Juice:** Al golpear hacia abajo sobre un enemigo o pincho (Pogo Jump), se instancia un efecto de chispas (Prefab temporal) y el personaje sale disparado hacia arriba.
- **Corrutinas:** Mecánica de *Concentración de Alma*: al mantener presionada la tecla de curación durante 1.5 segundos inmóvil mediante una corrutina, se gasta Alma en el modelo y se recupera una máscara de salud.
- **UI & Audio:** Visualización de las Máscaras de vida y orbe de Alma en el Canvas. Sonido metálico característico del aguijón al cortar.

Tema 10: Pac-Man 2D (Asignado a: Grupo 6)

- **Listas y Prefabs:** Generación inicial de la cuadrícula de puntos comestibles (Pac-Dots) instanciando Prefabs y guardándolos en una `List<GameObject>` para contar cuántos faltan devorar.
- **Corrutinas:** Al comer la Píldora de Poder (Power Pellet), se activa una corrutina de 6 segundos que pone a todos los fantasmas en modo vulnerable (color azul) y restablece su velocidad al expirar el tiempo.
- **UI & SceneManager:** Marcador numérico en pantalla con `TextMeshProUGUI` sumando puntos. Si Pac-Man come todos los puntos de la lista, se carga la pantalla de victoria con `SceneManager`.

9. Molde Didáctico de Referencia: La Corrutina con Parpadeo Visual

Para que no tengan dudas de cómo implementar la invulnerabilidad no bloqueante en su personaje:

```
using System.Collections;
using UnityEngine;

public class SaludVisualPuente : MonoBehaviour
{
    private SpriteRenderer sr;
    private bool esInvulnerable = false;

    private void Awake()
    {
        sr = GetComponent();
    }

    public void SufrirGgolpe(int cantidadDanio)
    {
        if (esInvulnerable) return; // Si esta en periodo de gracia, ignora el golpe

        StartCoroutine(RutinaInvulnerabilidad(1.5f));
    }

    private IEnumerator RutinaInvulnerabilidad(float duracionTotal)
    {
        esInvulnerable = true;
        float tiempoLimite = Time.time + duracionTotal;

        // Bucle temporal no bloqueante: parpadeo cada 0.1 segundos
        while (Time.time < tiempoLimite)
        {
            sr.color = new Color(1f, 1f, 1f, 0.2f); // Casi transparente
            yield return new WaitForSeconds(0.1f);  // Espera real sin colgar Unity
            sr.color = Color.white;                  // Vuelve a opaco
            yield return new WaitForSeconds(0.1f);
        }

        sr.color = Color.white;
        esInvulnerable = false;
    }
}
```

> **¡Muchos éxitos en la Fase 3!**
> Recuerden que ante cualquier traba técnica con Git, con las referencias de los prefabs o con la vinculación del Canvas, el equipo docente está disponible en el servidor de Discord de la cátedra para destrabar dudas. ¡A programar!